

UNIVERSITY OF BUEA

FACULTY OF ENGINEERING
AND TECHNOLOGY

DEPARTMENT OF COMPUTER
ENGINEERING



REPUBLIC OF CAMEROON

PEACE-WORK-
FATHERLAND

PAIX-TRAVAIL-PATRIE

Course Title: INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Task 6: Database Design and Implementation

Done By:

Ayuk Nestor ~ FE22A164

Bengha Paul ~ FE22A147

Ngong kong Steffy ~ FE22A262

Njuma Joseph ~ FE22A247

Oben Meg ~ FE22A288

Course Instructor:

Dr Valery Nkemeni.

TABLE OF CONTENTS

- 1. Data Elements
 - 1.1. A. User Data Elements
 - 1.2. B. Network Feedback Elements
 - 1.3. C. Feedback Detail Elements
 - 1.4. D. Device Information Elements
 - 1.5. E. User Settings Elements
- 2. Conceptual Design
 - 2.1. Main Entities and Attributes
 - 2.2. Relationships Between Entities
 - 2.3. Why This Design?
- 3. ER Diagram
 - 3.1. Purpose of the ER Diagram
 - 3.2. Diagram Structure
 - 3.3. Visual Representation
- 4. Database Implementation
 - 4.1. Choice of Database: MySQL
 - 4.2. Table Creation Based on ER Diagram
 - 4.2.1. User Table
 - 4.2.2. Feedback Table
 - 4.2.3. FeedbackDetail Table
 - 4.2.4. Device Table
 - 4.2.5. Settings Table
 - 4.3. Constraints and Normalization
 - 4.4. Data Types and Indexing
- 5. Backend Implementation
 - 5.1. Key Backend Functions
 - 5.2. Core Technologies Used
- 6. Connecting Database to Backend
 - 6.1. Detailed Data Flow
 - 6.2. Example of Flutter to Backend Integration
 - 6.3. Typical SQL Queries Executed in Backend
 - 6.4. Best Practices Implemented

INTRODUCTION

The NetSense application is designed to empower mobile network users by enabling them to share real-time experiences and performance data about their mobile connectivity. Through a combination of technical measurements and user feedback, NetSense aims to improve transparency, promote accountability among mobile network operators, and contribute to better service delivery.

This document provides a detailed overview of the system's data architecture and backend implementation. It covers the core data elements, conceptual database design, entity-relationship modeling, MySQL-based implementation, and the integration of backend services using Node.js. Each section has been structured to guide developers, database administrators, and stakeholders through the logic and infrastructure underpinning the NetSense application.

The objective is to ensure that the data layer of the application is robust, scalable, and aligned with best practices for usability, security, and performance.

1. DATA ELEMENTS

Data elements are the fundamental building blocks of the database. They represent the **individual pieces of information** the NetSense app needs to collect, store, and manipulate in order to function properly.

In the context of NetSense — an app designed to capture **real-time user experiences about mobile networks** — the data elements are derived directly from its key features: **user registration, feedback submission, performance reporting, and settings configuration**.

◆ A. USER DATA ELEMENTS

These elements are required to manage user accounts and authentication.

Field	Description	Purpose
user_id	Unique identifier for each user	Primary key
username	Chosen display name of the user	Display and personalization
email	User's email address	Used for login and verification
password	Encrypted password	Authentication and security
location	Geographical location of the user (optional)	Supports geo-based feedback

These elements enable functionalities like **Login/Sign-Up** and help link users to their feedback and settings.

◆ B. NETWORK FEEDBACK ELEMENTS

These capture technical performance data experienced by users.

Field	Description	Purpose
feedback_id	Unique ID for each feedback entry	Primary key
user_id	References the user who submitted the feedback	Links feedback to a user
signal_strength	Numeric value representing network strength	Measures signal quality
latency	Measured ping/delay in ms	Indicates responsiveness
data_speed	Upload/download speed in Mbps	Indicates network performance

This is essential for generating **performance dashboards** on the home screen and identifying **network patterns** over time.

◆ C. FEEDBACK DETAIL ELEMENTS

These provide **qualitative insights** that supplement the technical feedback.

Field	Description	Purpose
feedback_id	Foreign key linking to main feedback entry	Maintains relationship with technical feedback
rating	Star rating (1 to 5)	Measures user satisfaction
mood_emoji	Emoji chosen by user to represent feeling	Adds emotional context
comments	Optional text comment	Provides additional details or complaints
submitted_at	Timestamp of detailed feedback	Helps in matching with technical data

These elements support **Feedback Prompt** and **Feedback Form** screens in NetSense.

◆ D. DEVICE INFORMATION ELEMENTS

Used to capture information about the user's device to help with diagnostics and analytics.

Field	Description	Purpose
device_id	Unique ID for each device	Primary key
user_id	Foreign key linking to a user	Associates device with a user
device_model	Model name of the smartphone	Helps understand hardware influence

This helps identify if performance issues are **device-dependent**.

◆ E. USER SETTINGS ELEMENTS

These elements allow users to customize their app experience.

Field	Description	Purpose
setting_id	Unique ID for each setting record	Primary key
user_id	Foreign key to link settings to a user	Ensures settings are user-specific
notification_pre f	Boolean (true/false)	Enables or disables push notifications
theme	App theme preference (e.g., light/dark)	UI personalization

These settings power the **Settings screen** in the app and improve **user experience** and **accessibility**.

2. CONCEPTUAL DESIGN

The conceptual design of the NetSense database defines the logical structure of the app's data. It focuses on identifying the main entities, their attributes, and the relationships between them. This structure supports essential features such as user authentication, network feedback submission, and user settings management.

MAIN ENTITIES AND ATTRIBUTES

- **User**
- Attributes: user_id, username, email, password, location
- **Feedback**
- Attributes: feedback_id, user_id, signal_strength, latency, data_speed
- **FeedbackDetail**
- Attributes: feedback_id, rating, mood_emoji, comments
- **Device**
- Attributes: device_id, user_id, device_model
- **Settings**
- Attributes: setting_id, user_id, notification_pref, theme

◆ RELATIONSHIPS BETWEEN ENTITIES

The **conceptual relationships** define how these entities are connected logically:

- **User–Feedback Relationship**
- One user can submit **many feedback entries**.
- Type: **One-to-Many (1:N)**
- This supports the Feedback Form and Feedback Prompt features.
- **Feedback–FeedbackDetail Relationship**
- Each feedback has **one related qualitative detail**.
- Type: **One-to-One (1:1)**
- Combines raw performance data with user experience (stars/comments).
- **User–Device Relationship**
- A user may own and use **multiple devices**.
- Type: **One-to-Many (1:N)**
- Helps identify device-specific issues.
- **User–Settings Relationship**
- Each user has a **single set of settings**.
- Type: **One-to-One (1:1)**
- Supports personalization and feature toggles like dark mode or low-data mode.

WHY THIS DESIGN?

- **Data normalization:** No duplicate data across tables.
- **Scalability:** Allows adding more attributes/entities later (e.g., feedback location, telecom provider).

- **Modularity:** Each table is cleanly separated and can be queried or extended independently.
- **Usability:** Supports all NetSense features: account management, multi-device support, detailed feedback collection, and custom settings.

3. ER DIAGRAM

The **Entity-Relationship (ER) Diagram** serves as a blueprint for the implementation of the database by visually representing the structure of data used in the NetSense mobile application. It clearly illustrates how entities interact and ensures the relationships among data elements are correctly established before actual implementation.

PURPOSE OF THE ER DIAGRAM

The ER diagram is used to:

- Visualize the structure of the database system.
- Clarify the associations between data entities.
- Guide the creation of tables and keys during implementation.
- Ensure proper organization and referential integrity.

DIAGRAM STRUCTURE

In the ER diagram:

1. **Entities** are represented as rectangles.
2. **Relationships** are shown as lines connecting entities.
3. **Cardinality** (1:1, 1:N) is labeled on each relationship.
4. **Primary keys (PK)** and **foreign keys (FK)** are identified in each entity.

VISUAL REPRESENTATION

Here is a simple textual layout that reflects the visual ER diagram:

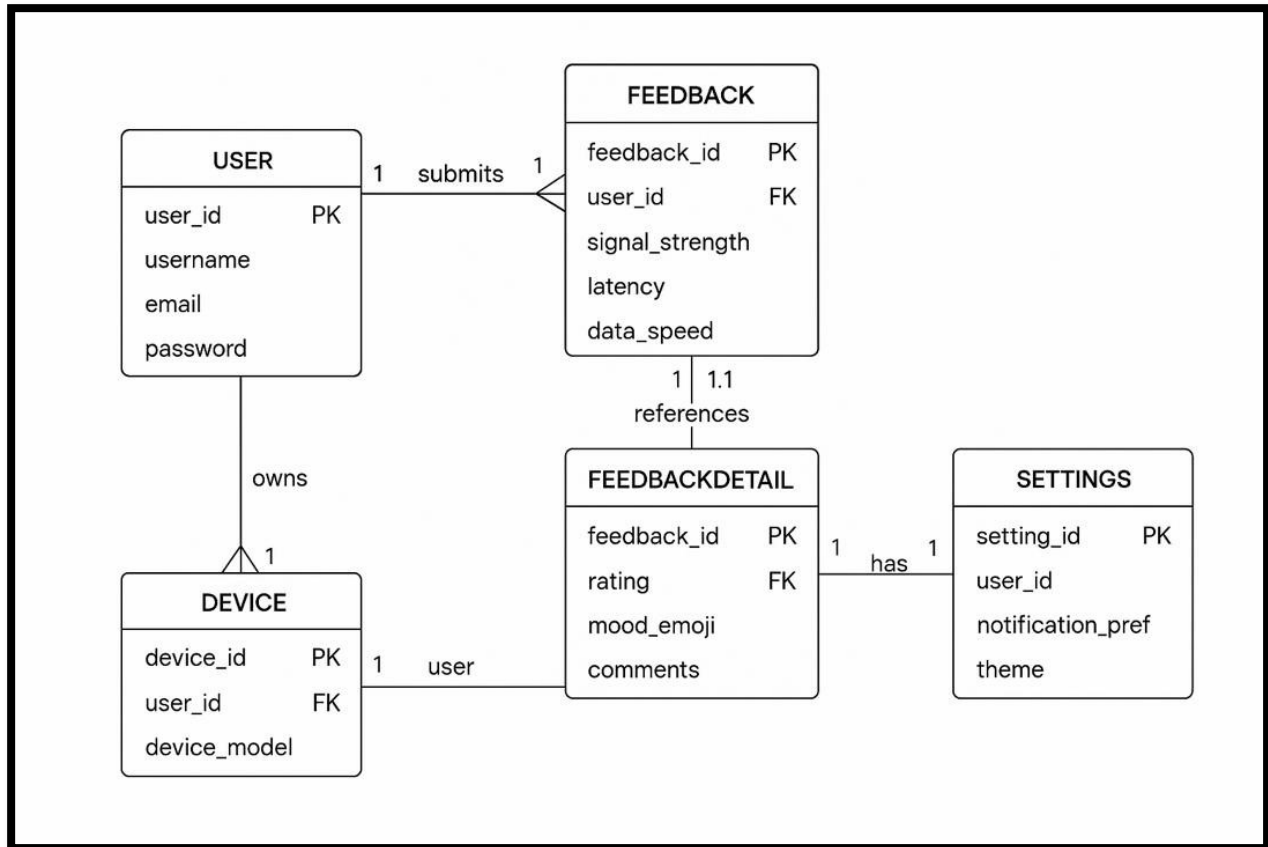


Fig 1.0

1. **User–Feedback**: One-to-Many (1:N)
2. **Feedback–FeedbackDetail**: One-to-One (1:1)
3. **User–Device**: One-to-Many (1:N)
4. **User–Settings**: One-to-One (1:1)

It ensures the backend and frontend interact with a **well-organized, normalized database**, capable of handling real-time feedback from mobile network users.

4. DATABASE IMPLEMENTATION

The **Database Implementation** phase involves transforming the ER diagram into a **fully functional MySQL database** using SQL commands. This step defines the **physical structure** of the database by creating tables, setting up relationships with **primary and foreign keys**, and applying constraints to ensure data integrity and reliability.

4.1. CHOICE OF DATABASE: MYSQL

MySQL was chosen as the database management system for the NetSense application due to its:

1. Open-source nature and wide availability.
2. Strong support for relational data.
3. Reliability in multi-user environments.
4. Compatibility with structured query language (SQL), making it suitable for handling user data and feedback efficiently.

4.2. TABLE CREATION BASED ON ER DIAGRAM

The following tables were implemented based on the ER diagram:

A. USER TABLE

```
CREATE TABLE User (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  location VARCHAR(100)
);
```

1. Stores registered users.
2. user_id is the primary key and auto-increments with each new user.

B. FEEDBACK TABLE

```
CREATE TABLE Feedback (
  feedback_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  signal_strength INT,
  latency FLOAT,
  data_speed FLOAT,
  FOREIGN KEY (user_id) REFERENCES User(user_id)
```

);

1. Stores network performance data submitted by users.
2. user_id acts as a foreign key linking feedback to the corresponding user.

C. FEEDBACKDETAIL TABLE

```
sql
CopyEdit
CREATE TABLE FeedbackDetail (
  feedback_id INT PRIMARY KEY,
  rating INT,
  mood_emoji VARCHAR(10),
  comments TEXT,
  FOREIGN KEY (feedback_id) REFERENCES Feedback(feedback_id)
);
```

1. Stores detailed user feedback (e.g., star rating, emoji, and comment).
2. Uses feedback_id as both a primary key and foreign key to enforce a one-to-one relationship.

D. DEVICE TABLE

```
sql
CopyEdit
CREATE TABLE Device (
  device_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  device_model VARCHAR(100),
  FOREIGN KEY (user_id) REFERENCES User(user_id)
);
```

1. Records the devices used by each user.
2. Supports multiple entries per user (one-to-many relationship).

E. SETTINGS TABLE

```
sql
CopyEdit
CREATE TABLE Settings (
  setting_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT UNIQUE,
  notification_pref BOOLEAN,
  theme VARCHAR(20),
  FOREIGN KEY (user_id) REFERENCES User(user_id)
);
```

1. Stores user-specific settings such as app theme and notification preferences.
2. user_id is unique to ensure one settings record per user (one-to-one).

4.3. CONSTRAINTS AND NORMALIZATION

1. The database is normalized to **Third Normal Form (3NF)**:
 1. Each table contains only data related to a single entity.
 2. Redundant data is minimized by using foreign keys.
2. **Constraints** such as NOT NULL, UNIQUE, and FOREIGN KEY enforce data validity and relationships.

4.4. DATA TYPES AND INDEXING

1. Data types are carefully chosen to balance storage and performance (e.g., VARCHAR for text, FLOAT for network speed).
2. Primary keys are **indexed by default**, and additional indexing can be applied to frequently searched fields like email.

CONCLUSION

The MySQL database for NetSense has been implemented using a well-structured, normalized schema based on the ER diagram. With properly defined tables, keys, and constraints, the database ensures consistent storage and easy retrieval of user feedback and profile information, supporting the core objectives of the application.

5. BACKEND IMPLEMENTATION

The backend of the NetSense mobile application was implemented using **Node.js** and the **Express.js** framework. It functions as the intermediary layer between the Flutter frontend (developed using Android Studio) and the MySQL database, ensuring all user actions are processed correctly and securely. The backend is responsible for handling HTTP requests, managing database operations, applying business logic, and returning structured responses.

Key Backend Functions:

1. **User Authentication:** When a user signs up or logs in through the Flutter app, the backend validates the credentials. Passwords are securely hashed using bcrypt and verified during login. Authenticated users receive a JWT (JSON Web Token), which is used to access protected resources.
2. **Feedback Management:** The backend receives user-submitted data related to network performance, such as signal strength, data speed, latency, and qualitative inputs like ratings and comments. These values are validated and stored in respective database tables. Each feedback entry is associated with the user's ID for future retrieval and analysis.
3. **Device and Settings Management:** The backend collects device information (e.g., device model, OS version) and stores user preferences like theme selection and notification settings. This supports app customization and helps in debugging device-specific issues.
4. **API Communication:** All communication between the Flutter frontend and the backend is done through RESTful APIs. These APIs accept and return data in JSON format. They include endpoints for user registration, login, submitting feedback, storing device data, and retrieving settings.

Core Technologies Used:

1. **Backend Framework:** Node.js with Express.js
2. **Frontend Framework:** Flutter (Dart) using Android Studio
3. **Database:** MySQL
4. **Authentication & Security:** JWT (for session handling), bcrypt (for password encryption)

This backend structure ensures that user actions are handled securely and efficiently, and that all data is stored in a well-structured, relational format.

6. CONNECTING DATABASE TO BACKEND

The backend connects to the MySQL database using the mysql2 library. This enables the application to perform essential database operations like inserting new records, updating user settings, and retrieving historical feedback data. The integration ensures that user-generated data from the mobile frontend is captured and stored accurately.

Detailed Data Flow:

1. **User Action on Flutter App:** The user performs an action, such as submitting feedback or changing settings.
2. **HTTP Request:** Flutter uses the http package to send a POST or GET request to a backend API endpoint.
3. **Request Handling in Backend:** The backend receives the request, validates the data, and constructs a SQL query.
4. **SQL Execution:** The query is executed on the MySQL database to insert, update, or fetch records.
5. **JSON Response:** The backend returns a response to the Flutter app indicating success or failure.

Example of Flutter to Backend Integration:

```
final response = await http.post(
  Uri.parse('http://your-server-ip:3000/api/feedback'),
  headers: {'Content-Type': 'application/json'},
  body: jsonEncode({
    "user_id": 1,
    "signal_strength": 4,
    "latency": 100,
    "data_speed": 5.2,
    "rating": 4,
    "comment": "Fair connection overall"
  }
);
```

Typical SQL Queries Executed in Backend:

1. Insert Feedback:

```
INSERT INTO Feedback (user_id, signal_strength, latency, data_speed)
VALUES (?, ?, ?, ?);
```

1. Insert Feedback Details:

```
INSERT INTO FeedbackDetail (feedback_id, rating, mood_emoji, comments)
VALUES (?, ?, ?, ?);
```

1. Get User Settings:

```
SELECT * FROM Settings WHERE user_id = ?;
```

Best Practices Implemented:

1. Use of parameterized SQL queries to prevent SQL injection.
2. Structured error handling with informative HTTP response codes.
3. Testing each API endpoint using tools like Postman before integration.
4. Consistent use of JSON format across all requests and responses.

This integration architecture ensures data consistency, enhances performance, and supports a smooth user experience on the Flutter app.